

Monitoreo de temperatura, humedad y movimiento en ESP32

Jeronimo Botero
Sara Muñoz
Luna Zapata
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	4
6. Explicación de funciones principales	4
7. Comunicación (si aplica)	5
8. Interfaz de usuario (si aplica)	5
9. Procedimiento de prueba	5
10. Manejo de Errores y Seguridad	6

1. Identificación del proyecto

- **Nombre del proyecto:** Monitoreo de temperatura, humedad y movimiento en ESP32 y microphyton
- **Jeronimo Botero**
Sara Muñoz
Luna Zapata

2. Resumen

Este proyecto consistió en un sistema de monitoreo IoT basado en un ESP32, diseñado para medir variables como la temperatura, la humedad y el movimiento en tiempo real, con el objetivo de visualizar los cambios en estas variables y detectar situaciones de peligro de manera rápida.

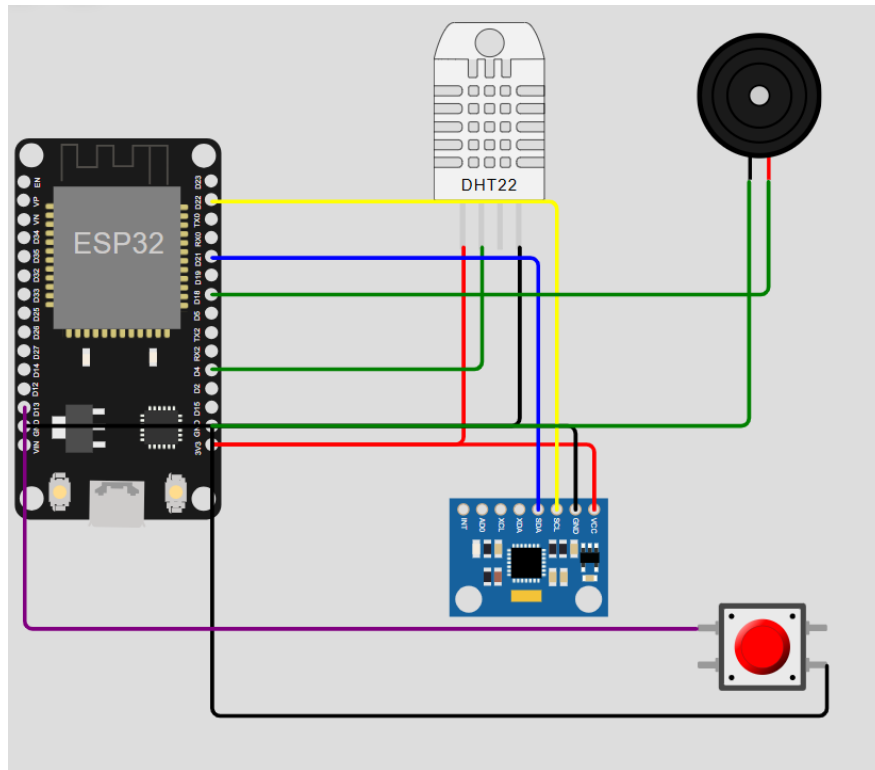
Para este proyecto utilizamos un sensor DHT22, encargado de detectar cambios en la temperatura y la humedad del entorno. También utilizamos un sensor de movimiento MPU6050 para identificar si hay movimientos bruscos, leves o si el sistema se encuentra en reposo. Además, se implementó un botón que funciona como alarma y envía las variables actuales por Telegram, junto con un buzzer que emite un sonido cuando se detectan cambios fuera de los umbrales establecidos o cuando se presiona el botón.

Por último, diseñamos una página web encargada de mostrar en tiempo real los valores de cada variable, indicando si alguno de los umbrales fue superado. Asimismo, el sistema envía mensajes por Telegram para alertar que alguna variable está fuera de rango y que es necesario tomar acción.

3. Descripción del hardware

- **ESP32** (modelo con ADC y pines GPIO)
- **sensor MPU6050:** conectado al pin 22 (SCL) y conectado al pin 21 (SDA)
- **DHT22:** conectado al pin 4
- **Boton alarma:** conectado al Pin 13
- **Buzzer:** conectado al Pin 18
- **Fuente de alimentación:** 5V (USB o regulada)

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** machine, time, DHT, unrequest, gc, socket, ujson, math
- **IDE:** Thonny

Flujo general del programa:

1. Se configuran los pines del ESP32 y se inician los sensores, el buzzer y el botón.
2. El ESP32 se conecta automáticamente a una red Wi-Fi.
3. Se inicia la lectura de los sensores DHT22 y MPU6050 para obtener temperatura, humedad y movimiento.
4. Se crea una página web para mostrar los datos en tiempo real.
5. El sistema revisa constantemente si las variables están dentro de los rangos permitidos.
6. Si alguna variable supera los umbrales, se activa una alarma con el buzzer y se envía un mensaje por Telegram.
7. El botón de pánico permite enviar una alerta manual junto con los valores actuales del sistema.

8. También se pueden consultar datos y umbrales desde Telegram mediante comandos sencillos.
9. Todo el sistema funciona continuamente mientras el ESP32 esté encendido.

5. Estructura del código

- Archivo principal: `main.py`
- No se usaron mdoulos externos, todo esta definidio dentro del mismo codigo
- El programa funciona dentro de un ciclo infinito (`while True`), donde constantemente se leen los sensores, se revisan alarmas, se reciben comandos de Telegram y se actualiza la página web.

6. Explicación de funciones principales

- **`conectar_wifi()`**: Se encarga de conectar el ESP32 a la red Wi-Fi y obtener la dirección IP para poder acceder a la página web.
- **`telegram()`**: Envía mensajes y alertas al chat de Telegram configurado en el sistema.
- **`mpu_init()`**: Inicializa el sensor MPU6050 para comenzar la lectura de movimiento.
- **`leer_mpu()`**: Lee los datos del sensor de movimiento y determina si el sistema está en reposo, en movimiento leve o en movimiento brusco.
- **`leer_sensores()`**: Obtiene los valores de temperatura, humedad y movimiento. Además, revisa si alguna variable supera los límites establecidos.
- **`beep()`**: Activa el buzzer para generar una alerta sonora cuando ocurre alguna alarma.
- **`verificar_boton()`**: Detecta cuando se presiona el botón de pánico y envía una alerta junto con los valores actuales del sistema.
- **`verificar_telegram()`**: Revisa si llegaron comandos desde Telegram y responde con la información solicitada.
- **`manejar()`**: Controla las solicitudes de la página web y envía los datos al navegador.
- **`iniciar_servidor()`**: Inicia el servidor web del ESP32 para mostrar la información en tiempo real.
- **`main()`**: Es la función principal del programa, donde se ejecuta todo el sistema y se mantiene funcionando continuamente.

7. Comunicación (si aplica)

Para este proyecto si utilizamos comunicacion vía WIFI para conectar las ESP32 con la pagina web y telegram. Podemos visualizar los datos de humedad, temperatura y movimiento en tiempo real.

Además, el sistema envía mensajes por Telegram cuando alguna variable se sale de los límites establecidos o cuando se presiona el botón de pánico. También se pueden consultar algunos datos del sistema por medio de comandos sencillos en Telegram.

8. Interfaz de usuario (si aplica)

La primera interfaz es una página web a la que el usuario puede acceder desde cualquier navegador, ya sea desde el celular o desde el computador. Para que funcione correctamente tanto el dispositivo desde el que se va a abrir la página como el ESP32 deben estar conectados a la misma red WiFi, de lo contrario la página no cargará, una vez dentro, el usuario puede ver en tiempo real los valores de temperatura, humedad y movimiento que están siendo medidos por los sensores, y saber de inmediato si alguno de esos valores está fuera del rango normal gracias a los indicadores de color que cambian entre verde y rojo según el estado del sistema y a las alarmas que detecta cuando no se está en el valor del umbral configurado.

La segunda forma de interacción es a través de un bot de Telegram, el sistema lo utiliza para enviar avisos automáticos al usuario, como por ejemplo, cuando el sistema se inicia correctamente, la dirección para abrir el navegador de la página web, cuando se detecta que la temperatura o la humedad están fuera de rango o que hubo un movimiento brusco, el bot envía un mensaje de alerta al instante, tambien cuando se presiona el botón físico conectado al ESP32 se envia una alerta al telegram, la cual indica que se activó el botón de pánico, además el usuario puede escribirle al bot directamente para recibir los valores actuales, si le manda la letra T recibe la temperatura, con H la humedad, con M el estado del movimiento y con U los límites de los umbrales configurados en el sistema.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Cargar el código usando thonny.
3. Ajustar los datos de configuración que están al inicio del código, los cuales son: el nombre de la red WiFi y su contraseña, el token del bot de Telegram y el ID del chat al que llegarán los mensajes.
4. Correr el programa, si todo está bien conectado, en la consola de Thonny se verá que el sistema se conecta correctamente al WiFi, muestra la dirección IP asignada, confirma que el sensor de movimiento MPU6050 respondió correctamente y que el servidor web quedó activo..

5. Verificar por medio de Telegram que llegaron los dos mensajes automáticos por medio del bot, uno confirmando que el sistema inició correctamente y otro con la dirección para abrir la página web.
6. Se abre esa dirección en el navegador y se comprueba que los tres datos (Temperatura, humedad y movimiento) aparecen con valores reales y que cambian dependiendo del entorno en el que se encuentren.
7. Verificar que las alarmas funcionen correctamente, cuando los valores obtenidos por los sensores sobrepasan los umbrales configurados, Temperatura (18°C - 28°C), Humedad (30 - 70) y Movimiento (Leve: 1.5 g y Brusco: 4.0 g), si esto pasa, la letra debe cambiar a color rojo e indicar que la alarma está activa, suena el buzzer y llega una notificación a Telegram.
8. También debe llegar una alerta correspondiente a Telegram cuando se presiona el botón de pánico.
9. Se puede escribir en el chat del Bot en Telegram, las letras T, H, M y U para que envíe los valores obtenidos por cada sensor en tiempo real, T (Temperatura), H (Humedad), M (Movimiento) y U (Umbrales configurados).

10. Manejo de Errores y Seguridad

El código está escrito de manera que si algo falla en alguno de los componentes, el sistema no se detiene por completo sino que registra el problema en la consola y sigue funcionando con lo que tiene disponible. Por ejemplo, si el sensor DHT22 no responde en alguna lectura, los datos de temperatura y humedad simplemente no se actualizan en ese momento, pero el resto del sistema continúa midiendo movimiento, atendiendo la página web y enviando algunos datos a Telegram.

Durante las pruebas se presentaron algunos inconvenientes relacionados con la conectividad. Aunque el computador y el ESP32 estaban conectados a la misma red WiFi, en algunos casos no se encontraban correctamente dentro de la misma dirección IP de la red, lo que impedía que la página web abriera desde el navegador.

También se presentaron problemas cuando la contraseña del WiFi no estaba escrita exactamente igual en el código, ya que cualquier error en una letra o alguna mayúscula hacía que el ESP32 no lograra conectarse.

Otra situación que generó dificultades fue una conexión a internet inestable o lenta. Esto ocurrió, por ejemplo, al momento de realizar la entrega del laboratorio, afectando tanto el envío de mensajes por Telegram como la carga de la página web, que en algunos casos se quedaba cargando indefinidamente sin mostrar los datos.

El sistema fue programado de tal forma que las alarmas solo se envían una vez cuando los valores obtenidos del sensor sobrepasan el umbral configurado. El mensaje aparece únicamente la primera vez y no en cada lectura del sensor, evitando así que el chat de Telegram se llene de mensajes repetidos y que el ESP32 se sature con demasiada información.

El botón físico (botón de pánico) tiene una configuración similar. Aunque se mantenga presionado, solo registra una acción cada cierto tiempo para evitar que se envíen múltiples reportes por un solo toque.